# 1º Entregable BBDD - ERS - CS2-SVSS_Jose-Angel-Alejo


- **Autor/Alumno**: JOSÉ ÁNGEL ALEJO SILLERO
- Instituto **Tecnológico Pablo de la Torre**


- **Curso**: Desarrollo de Aplicaciones Web 1º
- **Módulo**: Base de Datos
- **Documento**: Especificación de Requisitos de Software (ERS)
- **Título del proyecto**: **CS2-SVSS - Counter Strike 2 Stats Visualizing System**
- Fecha **de concepción de la idea**: **28 de Septiembre de 2025**
- Fecha de **creación**: **30 de Septiembre de 2025**
- Fecha de **última modificación**: **8 de Noviembre de 2025**

# CONTROL DE VERSIONES

| VERSIÓN | FECHA | CAMBIOS | AUTOR |
|---|---|---|---|
| 1.0 | 06/10/2025 | ERS a falta de revisión | José Ángel Alejo Sillero |

# ÍNDICE


- 1º Entregable BBDD - ERS - CS2-SVSS_Jose-Angel-Alejo
  - CONTROL DE VERSIONES
  - ÍNDICE
  - 1. INTRODUCCIÓN
    - 1.1 Propósito
    - 1.2 Alcance
    - 1.3 Definiciones, Acrónimos y Abreviaturas
  - 2. DESCRIPCIÓN GENERAL
    - 2.1 Descripción del negocio
    - 2.2 Descripción del problema
    - 2.3 Objetivos
  - 3. REQUISITOS FUNCIONALES
    - 3.1 Requisitos de almacenamiento de Datos
      - Entidades principales, sus atributos claves y las relaciones entre ellas:
      - Entidades futuras planificadas pero no implementadas en esta fase:
    - 3.2 Requisitos de integridad de datos
    - 3.3 Requisitos de Seguridad de Datos
    - 3.4 Requisitos de Mantenimiento de Datos
  - 4. CONCLUSION


---

# 1. INTRODUCCIÓN

## 1.1 Propósito

El propósito de este documento es **definir de manera clara y completa la especificación de requisitos de software** para el desarrollo del sistema de visualización de estadísticas de Counter Strike 2, siguiendo el **estándar IEEE Std. 830-1998**. Este documento servirá como **guía fundamental para el equipo de desarrollo**, garantizando que todos los **requisitos técnicos, funcionales y de seguridad** estén correctamente definidos y sean comprensibles para todas las partes involucradas en el proyecto.

El sistema propuesto tiene como finalidad ofrecer a los jugadores de Counter-Strike 2 una **plataforma web** donde puedan **consultar sus estadísticas de juego de forma automática y en tiempo real**, al mismo tiempo que proporciona a **Valve Corporation** un **sistema organizado y eficiente de gestión de reportes de conducta** para facilitar el trabajo de sus inspectores de **Overwatch**.

Este documento, elaborado como actividad práctica del módulo de Base de Datos de 1º de Desarrollo de Aplicaciones Web, aplica los conceptos del estándar IEEE 830-1998 al contexto específico de una **base de datos relacional** para un sistema de información vinculado al ecosistema competitivo de Counter-Strike 2.

Al proporcionar un marco bien documentado y estructurado, busco **minimizar los riesgos de vaguedades (o ambigüedades) durante el desarrollo** y asegurar que el sistema final cumpla con las expectativas tanto de los usuarios finales como de Valve Corporation y, por tanto, del equipo de desarrollo de Counter Strike 2 y su equipo de supervisión de trampas de jurado "Overwatch", estableciendo las bases para un **producto de calidad** que mejore la experiencia de la comunidad de jugadores.

## 1.2 Alcance

El sistema que se va a desarrollar recibirá el nombre técnico de **"CS2-SVSS - Counter Strike 2 Stats Visualizing System"**, aunque su nombre comercial de cara al público será **"FRAGIFY"**. El objetivo principal del sistema es doble: primera y principalmente **recopilar y almacenar de forma automática, eficiente y en tiempo real las estadísticas** de los jugadores de Counter-Strike 2 mediante el uso de una API gratuita con mayor precisión, y por otro lado, **facilitar a Valve Corporation el proceso de revisión de casos de mal comportamiento** a través de un sistema centralizado de reportes(por autentificación que funcionará a través de una página web (FRONTEND, mediante lenguajes de marcas)). Nuestro sistema backend emplea una **base de datos centralizada** que contendrá la siguiente información:

- **Información completa de los jugadores** registrados en Counter-Strike 2.
- **Estadísticas individuales de rendimiento** en partidas competitivas MM y PREMIERE de Valve.
- **Rankings competitivos de PREMIERE** actualizado(puntos en PREMIERE).
- **Datos detallados de partidas** jugadas.
- **Información de servidores de juego** donde se alojan las partidas(a través de nuestro API).
- Información completa de jugadores registrados en nuestra plataforma (FRAGIFY - PÁGINA WEB FRONTEND DE NUESTRO PROYECTO,) → **!No contemplado durante el desarrollo de nuestra base de datos** !Punto dependiente de la creación del la página web FRONTEND.
- **Sistema completo de reportes de conducta** que indica: → **!No contemplado durante el desarrollo de nuestra base de datos** !Punto dependiente de la creación del la página web FRONTEND.
    - El usuario reportado.
    - Que usuario de Steam(y por tanto de nuestra plataforma) ha hecho cada reporte.
    - Cuantos reportes tiene en total el usuario reportado.

    - Fecha en que se ha realizado cada reporte y ordenado de mas antigua a mas reciente(de arriba a abajo).
    - Partida asociada al reporte (a que partida hace referencia el reporte generado).
    - Fecha de la partida exacta.
    - Código de compartición de repetición de partida.

Para utilizar el sistema de reportes, los usuarios deberán sincronizar su perfil de Steam (mediante **Steam OAuth**) y proporcionar su **clave de juego única** para verificar que efectivamente la persona que realiza la sincronización es quien dice ser, garantizando así la **autenticidad de los reportes**. Además, al vincular su cuenta de Steam y proporcionar la clave de juego. Además, agregar que el sincronizar la cuenta de Steam y agregar la clave de juego única hace que nuestra Base de Datos tome la información directamente de los servidores de Valve conforme el usuario juega al juego, asegurando la **fiabilidad 100% de los datos** (pues ningún API gratuito de toma de datos de CS2 será 100% preciso). Para que Valve Corporation tenga acceso a los informes de reportes, se le proporcionará una **clave de acceso con permisos de solo lectura** a la sección de reportes de la base de datos, permitiéndoles acceder directamente a esta información organizada para sus inspecciones de Overwatch.

Sin embargo, nuestro sistema no contempla durante su desarrollo las siguientes áreas:

- Desarrollo y mantenimiento de nuestra página WEB FRONTEND (mediante lenguajes de marcas aún por especificar). Debido a esto, tampoco se incluyen los siguientes sistemas planteados anteriormente:
    - Sistema de reportes con datos almacenados en nuestra BBDD.
    - Sistema de reportes “FRAGIFY”, puesto que esta opción depende de la creación de nuestra página web, y por tanto:
        - Posibilidad de sincronizar cuenta de Steam mediante Steam OAuth, y posibilidad de enlazar clave de juego único.
- **Hosting o alojamiento de servidores de juego**: Nuestro sistema **NO** está orientado a gestionar ni alojar servidores de juego propios ni mostrar estadísticas de otras plataformas competitivas como FACEIT o ESL.
- **Gestión de sanciones o bans**: La aplicación de penalización de jugadores será competencia exclusiva de Valve Corporation y, por tanto de sus desarrolladores. Nuestro sistema **solo facilita la información organizada**.

## 1.3 Definiciones, Acrónimos y Abreviaturas

- **AES (Advanced Encryption Standard):** Algoritmo de cifrado simétrico ampliamente utilizado para proteger datos sensibles tanto en reposo como en tránsito.
- **API**: Application Programming Interface. Interfaz de programación de aplicaciones que permite la comunicación entre diferentes sistemas de software
- **Backup**: Proceso de copia y almacenamiento de los datos de un sistema en un medio seguro, con el propósito de proteger la información ante posibles pérdidas o fallos del sistema.
- **Baneo Overwatch**: Baneo aplicado por el jurado Overwatch. Aplica las mismas restricciones en una cuenta de Steam que un baneo de VAC. La diferencia es que el baneo VAC es irremovible, osea ser el baneo es definitivo y te impide jugar en cualquier juego con sistema VAC y hacer intercambios de objetos en todo Steam. Por otro lado un baneo Overwatch se puede apelar, y el baneo puede o no ser revisado (aunque el baneo sea apelado, esto no asegura que el equipo de desarrollo lo revise) por el equipo de desarrolladores de CS2, una o múltiples veces durante un tiempo no determinado, teniendo la posibilidad de que el baneo sea levantado, solo te impide jugar al juego donde se aplicó el baneo y, sólo te bloquea los intercambios de objetos del juego donde recibiste el baneo.
- **BBDD**: Base de datos. Sistema organizado de almacenamiento de información estructurada.

- **Clave de juego única:** Código de autenticación que vincula el juego Counter-Strike 2 con la cuenta de Steam de un usuario. Permite tener acceso directo a los datos de juego al completo de CS2(no limitados únicamente al modo Competitivo y al modo Premiere), facilitando la creación de estadísticas y datos no disponibles mediante APIs gratuitas.
- **Clave/Código de compartición de demos**: Clave gracias a la cual se puede mandar los datos de una Demo/Repetición, y al introducirla en el reproductor de repeticiones ingame, este interpreta los datos y te permite ver la Demo desde el punto de vista de cualquier jugador, no te fija a un punto de vista único (como sí ocurre con una grabación de repetición convencional), sin necesidad de mandar el archivo de video(causaría inflación innecesaria en nuestra BBDD).
- **Clutch:** Situación en Counter-Strike donde un jugador debe enfrentarse solo a múltiples enemigos tras la eliminación de sus compañeros de equipo.
- **CS2:** Counter-Strike 2. Videojuego multijugador de disparos tácticos en primera persona y orientados a competitivo, desarrollado por Valve Corporation.
- **CS2-SVSS:** Counter Strike 2 Stats Visualizing System.
- **CSRF (Cross-Site Request Forgery):** Tipo de ataque de seguridad que obliga a usuarios autenticados a ejecutar acciones no deseadas en aplicaciones web.
- **CT (Antiterroristas/Counter-Terrorists):** Uno de los dos equipos en Counter-Strike 2, cuyo objetivo es desactivar la bomba plantada por los terroristas o eliminarlos antes de que lo consigan.
- **Demo/Repetición**: Archivo de grabación de una partida de Counter-Strike 2 que permite reproducir exactamente todo lo ocurrido en una partida para su análisis posterior.
- **ELO:** Sistema de puntuación utilizado en el modo PREMIERE para clasificar a los jugadores según su habilidad. La puntuación aumenta con victorias y disminuye con derrotas, reflejando el nivel de juego del jugador.
- **ERS**: Especificación de Requisitos de Software.
- **Fragify**: Nombre comercial de la plataforma en WEB destinado al público general.
- **Griefing:** Comportamiento intencional de un jugador para sabotear, molestar o perjudicar a su propio equipo, como bloquear a compañeros, causar daño amigo deliberado o no colaborar.
- **Headshot:** Eliminación realizada mediante disparo a la cabeza del enemigo, que causa mayor daño y es considerada una métrica de habilidad muy importante.
- **IEEE 830-1998**: Estándar del Instituto de Ingenieros Eléctricos y Electrónicos para la especificación de requisitos de software.
- **Ingame**: Término coloquial con el que nos referimos a que lo que contamos sucede, está disponible o se desarrolla dentro del juego. Ha sido adaptado por desarrolladores, streamers y jugadores de la industria de los videojuegos, convirtiéndose en un término aceptado en entornos algo más técnicos.
- **JWT (JSON Web Token):** Estándar abierto que permite crear tokens de acceso que certifican la identidad de usuarios y permiten autenticación segura entre cliente y servidor.
- **K/D Ratio**: Kill/Death Ratio. Estadística que muestra la relación entre eliminaciones realizadas y muertes sufridas por un jugador.
- **KDBX:** Formato de archivo de base de datos de KeePass utilizada para almacenar contraseñas y credenciales de forma cifrada y segura.
- **MM (Matchmaking):** Sistema de emparejamiento automático de Counter-Strike 2 que agrupa jugadores de nivel similar para partidas competitivas por mapas específicos.
- **Modelo ACID**: Modelo que establece una serie de **principios** durante el desarrollo de un sistema de BBDD, los cuales son imprescindibles en sistemas donde los datos son valiosos y los errores costosos.
- **MR12 (Max Rounds 12):** Sistema actual de Counter-Strike 2 donde el primer equipo en ganar 13 rondas gana la partida en modo regulación.

- **MR15 (Max Rounds 15):** Sistema antiguo de Counter-Strike donde el primer equipo en ganar 16 rondas ganaba la partida.
- **MVP (Most Valuable Player):** Reconocimiento otorgado al mejor jugador de una ronda específica en Counter-Strike 2, basado en su contribución al equipo.
- **OAuth:** Protocolo de autorización abierto que permite autenticación segura de usuarios sin necesidad de compartir contraseñas directamente con aplicaciones de terceros.
- **Overwatch**: Sistema de revisión de Valve Corporation (específicamente pertenece al sistema de desarrollo de CS2), en el cual un jurado formado por personal de Valve Corporation revisa reportes de conducta inapropiada en partida, los cuales pueden haber pasado desapercibido por el sistema anticheat VAC, aplicando en caso de un veredicto culpable un baneo de Overwatch en el perfil del jugador.
- **Principios ACID**: Hacen referencia al Modelo ACID. Son:
  - **Atomicidad**: El todo o nada de una transacción.
  - **Consistencia**: Manteniendo la integridad de los datos.
  - **Aislamiento**: Transacciones independientes entre sí.
  - **Durabilidad**: Persistencia a largo plazo.
- **RAID (Redundant Array of Independent Disks):** Tecnología de almacenamiento que combina múltiples discos duros para proporcionar redundancia de datos, tolerancia a fallos y/o mejora de rendimiento.
- **Ranking competitivo**: Sistema de clasificación de jugadores según su nivel de habilidad en Counter-Strike 2, que va desde Silver hasta Global Elite, y que desde la salida del modo PREMIERE solo se aplica por mapa. No se tiene en cuenta en las tablas de clasificación global.
- **Ranking PREMIERE**: Sistema de clasificación de habilidad en Counter-Strike 2, es independiente al modo PREMIERE. La puntuación va desde 0 de ELO(o puntos) hasta 30K de ELO(por el momento, pues no está capado por un puntaje máximo), y se divide por grupos. Es el que más valoran los jugadores a día de hoy. Se tiene en cuenta en las tablas de clasificación GLOBAL, REGIONAL y por AMIGOS.
- **RGPD**: Reglamento General de Protección de Datos. Normativa europea sobre protección de datos personales.
- **RPO (Recovery Point Objective):** Cantidad máxima aceptable de pérdida de datos medida en tiempo tras un incidente o fallo del sistema.
- **RTO (Recovery Time Objective):** Tiempo máximo aceptable que un sistema puede estar fuera de servicio tras un incidente antes de que se considere crítico para el negocio.
- **Soft Delete:** Técnica de eliminación lógica donde los registros se marcan como eliminados mediante un campo de estado, pero no se borran físicamente de la base de datos, permitiendo recuperación si es necesario.
- **SSH (Secure Shell):** Protocolo de red criptográfico que permite acceso remoto seguro a sistemas y servidores, utilizado para administración y transferencia de archivos.
- **Steam**: Plataforma de distribución digital de videojuegos desarrollada por Valve Corporation.
- **SteamID64**: Identificador único de la cuenta de Steam de cada usuario en la plataforma Steam, en formato de 64 bits, usado por la Steam Web API, SteamDB y otras herramientas.
- **T (Terroristas):** Uno de los dos equipos en Counter-Strike 2, cuyo objetivo es plantar la bomba en uno de los sitios designados o eliminar al equipo contrario antes de que acabe el tiempo.
- **Tier/Grupo de clasificación:** Categorías o rangos dentro del sistema PREMIERE que agrupan jugadores según sus puntos de ELO, facilitando la visualización del nivel general del jugador.
- **TLS (Transport Layer Security):** Protocolo criptográfico que proporciona seguridad y privacidad en las comunicaciones por internet, sucesor de SLL.
- **VAC (Valve Anticheat System)**: Sistema Anticheat de Valve Corporation, se aplica a todos los juegos online desarrollados por Valve Corporation.

- **XSS (Cross-Site Scripting):** Vulnerabilidad de seguridad web que permite a atacantes inyectar scripts maliciosos en páginas web vistas por otros usuarios.

---

# 2. DESCRIPCIÓN GENERAL

En esta sección se ofrece una visión integral del proyecto, abordando el contexto en el que se inserta el sistema, las limitaciones actuales que experimentan tanto los desarrolladores que desean trabajar con datos de Counter-Strike 2 como los jugadores que buscan visualizar sus estadísticas, y los objetivos específicos que guiarán el desarrollo de la base de datos backend. Esta descripción general es fundamental para asegurar que el sistema de almacenamiento a desarrollar esté completamente alineado con las necesidades identificadas y que establezca las **bases sólidas para futuras implementaciones** como la plataforma web **"Fragify"** y el **sistema de reportes**.

## 2.1 Descripción del negocio

Counter-Strike 2 es uno de los **videojuegos competitivos más populares del mundo**, con millones de jugadores activos diariamente que participan en partidas clasificatorias en los modos **Competitivo MM (Matchmaking)** y **PREMIERE**. El ecosistema de Counter-Strike 2 se sustenta en la plataforma **Steam de Valve Corporation**, donde cada jugador posee un **perfil único vinculado a su SteamID**, y una **clave de juego única** vinculada a su perfil de CS2 y, por tanto, a dicho SteamID, que almacena su progreso, estadísticas y rankings competitivos, siendo el papel de la clave única de juego acceder a la información del usuario asociada al SteamID.

Actualmente, los jugadores acceden a sus estadísticas básicas directamente desde el cliente del juego o desde su cliente/perfil de Steam, pero esta información es **limitada y no ofrece un análisis detallado del rendimiento**. Existen plataformas de terceros que intentan ofrecer estadísticas más completas mediante APIs gratuitas, pero estas APIs tienen **limitaciones importantes** en cuanto a precisión, actualización en tiempo real y acceso a datos completos del juego. Por otro lado, los desarrolladores independientes que desean crear herramientas o análisis basados en datos de Counter-Strike 2 enfrentan la dificultad de **no contar con una base de datos estructurada y confiable** que centralice esta información.

El proyecto CS2-SVSS nace con la intención de crear una **base de datos backend robusta y centralizada** que almacena de forma estructurada y eficiente toda la información relevante sobre jugadores, estadísticas de rendimiento en los modos MM y Premiere, rankings competitivos (especialmente el **sistema de ELO de PREMIERE** que es el más valorado por los jugadores en la actualidad), datos detallados de partidas y servidores de juego. Esta base de datos será el **núcleo** que posteriormente permitirá el desarrollo de **Fragify**, la plataforma web donde los jugadores podrán visualizar sus estadísticas, y también incorporará un **sistema de reportes de conducta** que facilitará el trabajo a los inspectores de **Overwatch** del equipo de desarrollo de CS2 en Valve Corporation.

Los actores principales implicados en el proyecto actual son los desarrolladores del sistema CS2-SVSS, que diseñarán e implementarán la estructura de la base de datos, los administradores que mantendrán la integridad y seguridad de los datos almacenados, y potencialmente Valve Corporation como colaborador futuro una vez se implemente el sistema de reportes. En fases posteriores, se incorporarán los jugadores de Counter-Strike 2, como usuarios finales de la plataforma web Fragify.

## 2.2 Descripción del problema

Actualmente, tanto los desarrolladores que desean trabajar con datos de Counter-Strike 2 como los jugadores que buscan análisis detallados de su rendimiento enfrentan varias limitaciones significativas que justifican el desarrollo de este sistema de base de datos.

1. **En cuanto al acceso y almacenamiento de datos**:
    1. Existe una **carencia de bases de datos centralizadas y estructuradas** que recopilen de forma organizada la información completa de CS2. Las APIs gratuitas disponibles presentan limitaciones

importantes:

1. **No son completamente precisas al 100%**.
2. Tienen **restricciones en la frecuencia de actualización**.
3. **No proporcionan acceso a todos los modos de juego** (limitándose generalmente a Competitivo y PREMIERE).
4. **Carecen de datos históricos completos** que permitan análisis de evolución a largo plazo.

2. Esta **fragmentación y falta de fiabilidad** dificulta que desarrolladores independientes puedan crear herramientas de análisis, sistemas de seguimiento o plataformas de estadísticas con datos confiables.

2. **Respecto a la estructura y organización de la información**:
    1. Los datos obtenidos de diferentes fuentes suelen estar **desorganizados**, en **formatos inconsistentes** y sin una **estructura relacional** que permite consultas eficientes.
    2. **No existe una normalización clara** de entidades como jugadores, partidas, estadísticas, rankings y servidores, lo que complica la implementación de sistemas que necesiten cruzar información entre estas entidades.
    3. La ausencia de una base de datos bien diseñada con **integridad referencial**, **validaciones adecuadas** y **optimización de consultas** representa un obstáculo importante.
3. **Desde la perspectiva de los jugadores**:
    1. Aunque este proyecto se centra inicialmente en el backend, la problemática que se busca resolver a futuro es la falta de acceso centralizado y detallado a información de rendimiento.
        1. El cliente oficial de CS2 muestra únicamente estadísticas básicas agregadas sin ofrecer análisis de tendencias, comparativas detalladas entre mapas, evolución del ranking PREMIERE a lo largo del tiempo, o métricas avanzadas como precisión por arma específica, ratio de headshots, rendimiento en situaciones de clutch o análisis de posicionamiento en el mapa.
    2. Esta carencia de información detallada y bien presentada dificulta que los jugadores identifiquen áreas de mejora y puedan progresar de forma consciente.
4. **En relación al futuro sistema de reportes planificado**:
    1. Actualmente existe una **falta de transparencia y organización** en el proceso de reporte de conductas inapropiadas.
        1. Los jugadores pueden reportar desde el juego, pero **no tienen seguimiento sobre sus reportes** ni pueden aportar información adicional de manera estructurada.
        2. Desde la perspectiva de Valve, el **volumen masivo de reportes** recibidos diariamente dificulta su proceso eficiente, ya que los inspectores de Overwatch deben **buscar manualmente las partidas** en sus sistemas internos. La ausencia de un sistema que centralice y organice los reportes con toda la información necesaria (código de compartición de demo, fecha exacta, usuarios involucrados, historial), representa una oportunidad de mejora que este proyecto contempla para fases futuras.

> **CONCLUSIÓN**: **La falta de una base de datos backend sólida, estructurada y escalable es el problema fundamental que este proyecto aborda**, estableciendo los cimientos necesarios para que posteriormente se puedan desarrollar las funcionalidades de visualización web y gestión de reportes de forma eficiente y confiable.

## 2.3 Objetivos

Como ya establecimos anteriormente, el **objetivo principal** del desarrollo de la BBDD CS2-SVSS es **crear un sistema backend robusto, eficiente y escalable** que centralice y estructure toda la información relevante de Counter-Strike 2, estableciendo las bases sólidas para el futuro desarrollo de la plataforma web Fragify y el sistema

de reportes, cumpliendo en todo momento con los **"Principios del Modelo ACID"**, ofreciendo **seguridad, integridad y confiabilidad** a las operaciones en bases de datos.

Para lograr esto, se establecen los siguientes objetivos específicos, todos ellos medibles y verificables:

1. **Respecto al diseño y estructura de la base de datos**:
    1. Se busca crear un modelo relacional normalizado que organice de forma lógica y eficiente todas las entidades del sistema:
        1. Jugadores registrados en CS2
        2. Estadísticas individuales de rendimiento.
        3. Rankings competitivos de PREMIERE con sistema de ELO/puntos
        4. Datos detallados de partidas en los modos MM y PREMIERE.
        5. Información de servidores de juego donde se alojan las partidas.
        6. Estructura preparada para el futuro sistema de reportes, aunque esto no se implemente en la fase actual.
    2. El **indicador de éxito** será que el **modelo entidad-relación** esté completamente documentado, cumpla con la **tercera forma normal** para evitar redundancias, y todas las relaciones entre entidades estén claramente definidas con **claves primarias y foráneas** apropiadas.
2. **En cuanto a la recopilación y almacenamiento de datos**:
    1. El objetivo es implementar mecanismos que permitan obtener información de Counter-Strike 2 mediante **APIs gratuitas** disponibles, y posteriormente mediante integración directa con **Steam OAuth** y **claves de juego únicas** para lograr **precisión del 100%** en los datos (aunque esta última funcionalidad dependerá del desarrollo futuro del FRONTEND).
    2. El sistema debe ser capaz de almacenar **estadísticas detalladas** que incluyan:
        1. **Métricas básicas** como: Eliminaciones, muertes, asistencias y puntuación MVP.
        2. **Métricas avanzadas** como:
            1. Precisión por arma.
            2. Ratio de headshots.
            3. Daño promedio por ronda.
            4. Evolución histórica del ranking PREMIERE del jugador con sus puntos de ELO.
    3. El **indicador de éxito** será que la base de datos pueda almacenar de forma consistente datos de **al menos 50 a 100 jugadores** con sus historiales completos de partidas **sin degradación del rendimiento** en las consultas.
3. **Para garantizar la integridad y calidad de los datos**:
    1. Se debe implementar un conjunto completo de **restricciones, validaciones y reglas de negocio** que aseguren que la información almacenada sea **consistente, precisa y confiable**. Esto incluye:
        1. **Validación de datos y formatos específicos de CS2** (Steam IDs, códigos de compartición de demos, rangos de ELO de PREMIERE)
        2. **Unicidad de campos críticos**
        3. **Integridad referencial** entre todas las tablas
        4. **Triggers automáticos** que mantengan la consistencia de datos calculados y agregados.
    2. El **indicador de éxito** será que el **100% de los intentos de insertar datos inválidos o inconsistentes** sean rechazados por el sistema de validación, y que las **auditorías periódicas** de integridad no encuentren datos huérfanos ni inconsistencias
4. **Respecto a la preparación para funcionalidades futuras**:
    1. Aunque el sistema de reportes y la plataforma web Fragify no se implementarán en esta fase por depender del desarrollo del FRONTEND, la BBDD debe diseñarse considerando estos componentes. Se debe incluir en el diseño:

        1. La estructura de tablas necesaria para almacenar reportes de conducta con todos sus atributos:
            1. Usuario reportado.
            2. Reportante.
            3. Partida asociada.
            4. Fecha.
            5. Código de demo.
            6. Tipo de infracción.
            7. Estado de reporte.
        2. Modelo de permisos diferenciados que permitirá en el futuro que Valve Corporation acceda con permisos de solo lectura a la sección de reportes mediante una clave API.
    2. El indicador de éxito será que cuando se implemente el FRONTEND, la incorporación del sistema de reportes requiera únicamente activar las tablas ya diseñadas sin necesidad de modificaciones estructurales importantes en la BBDD
5. **En términos de rendimiento y escalabilidad**:
    1. El sistema debe optimizarse para soportar un **crecimiento considerable en el volumen de datos**.
    2. La BBDD debe ser capaz de gestionar **consultas complejas** que crucen información entre múltiples tablas (jugadores, partidas, estadísticas, ranking) con **tiempos de respuesta inferiores a 3s** incluso con miles de registros.
    3. Deben implementar **índices apropiados** en campos de búsqueda frecuente como Steam ID, fecha de partidas y ranking PREMIERE.
    4. El **indicador de éxito** será que el **90% de las consultas típicas del sistema** se completen en **menos de 1.5 segundos**, y que la base de datos pueda crecer hasta **200 partidas almacenadas** manteniendo estos tiempos de respuesta.
6. **Finalmente, establecer las bases de seguridad y cumplimiento normativo desde el inicio del proyecto:**
    1. Aunque no se implementarán todos los mecanismos de autenticación en esta fase (ya que dependen del FRONTEND):
        1. La BBDD debe diseñarse con un modelo de control de acceso basado en roles preparado para el futuro, donde existirán:
            1. Permisos diferenciados para administradores del sistema.
            2. El servicio API de actualización automática.
            3. Inspectores de Valve (solo lectura de reportes).
            4. Jugadores usuarios de Fragify.
        2. En todo momento se debe garantizar el cumplimiento del RGPD en cuanto al almacenamiento de datos personales, implementando desde el diseño mecanismos que faciliten en el futuro el derecho al olvido y la exportación de datos
    2. El indicador de éxito será que el modelo de seguridad esté completamente documentado y que los datos sensibles se identifiquen claramente para su futuro cifrado

> **CONCLUSIÓN**: Podemos observar cómo estos objetivos están directamente alineados con los problemas identificados y que proporcionan una hoja de ruta clara para el desarrollo de una BBDD backend sólida que servirá como fundamento del proyecto CS2-SVSS, permitiendo en fases posteriores la implementación eficiente de la plataforma web Fragify y el sistema colaborativo de reportes con Valve Corporation.

# 3. REQUISITOS FUNCIONALES

Este apartado detalla los requisitos funcionales que la BBDD debe cumplir para garantizar el correcto funcionamiento del sistema CS2-SVSS backend y establecer las bases para futuras implementaciones. Los requisitos se organizan en **cuatro categorías fundamentales** fieles a los **Principios ACID:**

- **Almacenamiento de datos**
- **Integridad de datos**
- **Seguridad de datos**
- **Mantenimiento de datos**

> Es importante destacar que algunos elementos descritos corresponden a **funcionalidades futuras planificadas pero no implementadas en esta fase del proyecto**, especialmente aquellas relacionadas con el FRONTEND y el sistema de reportes, pero se incluyen en el diseño de la BBDD para facilitar su incorporación posterior sin modificaciones estructurales importantes.

## 3.1 Requisitos de almacenamiento de Datos

La BBDD del sistema CS2-SVSS debe **gestionar y almacenar de forma estructurada** toda la **información crítica necesaria** para el funcionamiento actual del backend y la futura plataforma Fragify.

**Entidades principales, sus atributos claves y las relaciones entre ellas:**

1. **Entidad JUGADORES_CS2**: Esta entidad almacenará la **información básica de cada jugador de CS2** cuyos datos se recopilen del juego.
    1. Los **atributos** incluyen:
        1. El **identificador único** del jugador en nuestra base de datos (**ID_Jugador_CS2**).
        2. El **Steam ID único** del jugador que es **obligatorio** y debe ser **único** en toda la tabla.
        3. El **nombre del usuario actual de Steam** (que puede cambiar con el tiempo).
        4. La **fecha del primer registro** de datos del jugador en nuestro sistema.
        5. La **fecha de última actualización** de sus datos.
        6. Campos adicionales como la **región geográfica principal** donde juega.
    2. Esta entidad se **relaciona** con las entidades de estadísticas, partidas y rankings. Es importante notar que esta tabla almacena jugadores del juego CS2 en general, no usuarios registrados específicamente en la futura plataforma Fragify, esa sería una entidad diferente para fase futura.
2. **Entidad ESTADISTICAS_JUGADOR**: Contendrá las **métricas agregadas de rendimiento** para cada jugador.
    1. Los **atributos** principales son:
        1. El **identificador único** de la estadística (**ID_Estadistica**).
        2. La **clave foránea** que referencia al jugador (**ID_Jugador_CS2**).
        3. El **total acumulado de eliminaciones**.
        4. El **total de muertes**.
        5. El **ratio K/D** calculado automáticamente.
        6. El **total de Asistencias**.
        7. El número de **MVPs** obtenidos.
        8. La **precisión general de disparo** expresada en porcentaje.
        9. El **ratio de Headshots** sobre el total de eliminaciones.
        10. El **daño promedio causado por ronda**.
        11. El **total de partidas jugadas** en modos MM y PREMIERE.
        12. El **total de partidas ganadas**.
        13. El **porcentaje de victorias** calculado.
        14. La **fecha de última actualización** de estas estadísticas.
    2. Esta entidad tiene una **relación uno a uno** con **JUGADORES_CS2** para las estadísticas generales actuales, aunque también se contempla almacenar instantáneas históricas para análisis de evolución.

3. **Entidad RANKING_PREMIERE**: Dado que el **sistema de ranking PREMIERE con puntos de ELO** es el más valorado por los jugadores, esta entidad almacenará específicamente esta información.
    1. Los **atributos** son:
        1. **Identificador único** del registro (**ID_Ranking_PREMIERE**).
        2. La **clave foránea** del jugador (**ID_Jugador_CS2**).
        3. Los **puntos actuales de ELO en PREMIERE** (que pueden ir desde 0 hasta 30k o superior).
        4. El **grupo o TIER de clasificación** correspondiente a esos puntos.
        5. La **posición en la tabla de clasificación global** si está disponible.
        6. La **posición en la clasificación regional**.
        7. El **número total de partidas PREMIERE** jugadas.
        8. La **fecha de última actualización** del ranking.
    2. Esta entidad se relaciona con **JUGADORES_CS2** en una **relación uno a uno** para el ranking actual.
4. **Entidad RANKING_COMPETITIVO_POR_MAPA**: Aunque el ranking competitivo (de Silver a Global Elite) ha perdido relevancia desde que se estableció a por mapa en lugar de general y la consecuente introducción de PREMIERE, algunos jugadores aún le dan importancia.
    1. Los **atributos** son:
        1. El **identificador del registro** (**ID_Ranking_Mapa**).
        2. La **clave foránea** del jugador (**ID_Jugador_CS2**).
        3. El **nombre del mapa específico** (Dust 2, Mirage, Inferno, Nuke, Overpass, Vértigo, Ancient, Anubis).
        4. El **rango competitivo** en ese mapa específico (de **Silver I** hasta **Global Elite** o "No_Rank" en caso de no tener rango asignado).
        5. El **número de victorias** en ese mapa.
        6. La **fecha de última actualización**.
    2. Un jugador puede tener **múltiples registros** en esta tabla, uno por cada mapa en el que haya jugado competitivo MM.
    3. Se establece una **relación uno a muchos** entre **JUGADORES_CS2** y **RANKING_COMPETITIVO_POR_MAPA**.
5. **Entidad PARTIDAS**: Almacenará **información detallada de cada partida** jugada en los modos MM y PREMIERE.
    1. Los **atributos** son:
        1. El **identificador único** de la partida en nuestra base de datos (**ID_Partida**).
        2. Un **identificador único de partida** proporcionado por Valve si está disponible.
        3. El **código de compartición de demo**, que es único y permite descargar la repetición de la partida y su consecuente visualización, así como extracción de datos de la misma.
        4. La **fecha y hora exacta de inicio** de la partida.
        5. La **duración total en minutos**.
        6. El **mapa** donde se jugó.
        7. El **modo de juego específico** (Competitivo MM o PREMIERE).
        8. El **resultado final** expresado como puntuación (por ejemplo **13-11** en el nuevo sistema de CS2 llamado **MR12** o, **16-14** en formato antiguo llamado **MR15**).
        9. El **servidor** donde se alojó la partida (**ID_Servidor** como clave foránea).
        10. Campos adicionales como la **versión del juego** en ese momento.
    2. Esta entidad se relaciona con **JUGADORES_CS2** mediante una **tabla intermedia** que registra la participación de múltiples jugadores en cada partida (**ID_Partida_Jugador**), estableciendo una **relación muchos a muchos**, pues un jugador puede participar en múltiples partidas y una partida tiene múltiples jugadores.

3. Se relaciona con **SERVIDORES** mediante clave foránea, estableciendo una **relación uno a muchos**.

6. **Entidad PARTIDA_JUGADOR**: **Tabla intermedia** que registra la participación de cada jugador en cada partida específica.
    1. Los **atributos** son:
        1. **Identificador único** del registro (**ID_Partida_Jugador**).
        2. **Claves foráneas** del jugador (**ID_Jugador_CS2**) y la partida (**ID_Partida**).
    2. Esta entidad establece la **relación muchos a muchos** entre **JUGADORES_CS2** y **PARTIDAS**.
7. **Entidad SERVIDORES**: Contendrá **información sobre los servidores de juego oficiales** de Valve donde se alojan las partidas.
    1. Los **atributos** son:
        1. **Identificador único** del servidor (**ID_Servidor**).
        2. **Dirección IP** del servidor.
        3. **Región geográfica** del servidor (Europa Oeste, Europa Este, América del Norte, América del Sur, Asia, etc.)
        4. **Proveedor de la infraestructura** del servidor.
        5. **Latencia promedio** observada en milisegundos basada en mediciones de partidas.
        6. **Estado actual** del servidor (activo, mantenimiento, inactivo).
        7. **Fecha de última verificación** del estado del servidor.
    2. Esta entidad se relaciona con **PARTIDAS** en una **relación uno a muchos** ya que un servidor aloja múltiples partidas a lo largo del tiempo.
8. **Entidad RENDIMIENTO_PARTIDA**: Esta entidad almacena el **rendimiento específico de cada jugador** en cada partida individual.
    1. Los **atributos** son:
        1. El **identificador único** del registro (**ID_Rendimiento**).
        2. **Claves foráneas** del jugador (**ID_Jugador_CS2**) y la partida (**ID_Partida**).
        3. **Eliminaciones** realizadas en esa partida específica.
        4. **Muertes** sufridas.
        5. **Asistencias**.
        6. **Daño total causado**.
        7. **Precisión de disparo** en esa partida.
        8. Número de **headshots**.
        9. Si obtuvo **MVP** en alguna ronda.
        10. **Equipo** en el que jugó en cada ronda (**Terroristas/T** o **Antiterroristas/CT**).
        11. Si **abandonó la partida** antes de tiempo.
    2. Esta entidad establece la **relación muchos a muchos** entre **JUGADORES_CS2** y **PARTIDAS**.
9. **Entidad HISTORICO_RANKING_PREMIERE**: Para permitir el **análisis de la evolución del ranking PREMIERE** a lo largo del tiempo, esta entidad almacenará **instantáneas periódicas**.
    1. Los **atributos** son:
        1. **Identificador único** del registro histórico (**ID_Historico_PREMIERE**).
        2. La **clave foránea** del jugador (**ID_Jugador_CS2**).
        3. Los **puntos de ELO** en ese momento específico.
        4. El **grupo de clasificación** correspondiente.
        5. La **fecha de captura** de la instantánea.
        6. Número de **partidas PREMIERE** jugadas hasta ese momento.
    2. Esta entidad se relaciona con **JUGADORES_CS2** en **relación uno a muchos**, ya que un jugador tiene múltiples registros históricos que permiten visualizar su progresión.

**Entidades futuras planificadas pero no implementadas en esta fase:**

1. **Entidad USUARIOS_FRAGIFY (Futura):** Esta entidad almacenará **información de usuarios registrados** específicamente en la plataforma web Fragify cuando se desarrolle en el FRONTEND.
    1. Los **atributos** son:
        1. El **identificador único** del usuario en Fragify (**ID_Usuario_Fragify**).
        2. La **clave foránea** que vincula con el jugador de CS2 (**ID_Jugador_CS2**) que será **opcional** inicialmente pero **obligatoria** una vez el usuario sincronice su cuenta de Steam mediante OAuth.
        3. **Email de contacto** del usuario.
        4. **Contraseña hasheada** para autenticación.
        5. **Fecha de registro** en la plataforma Fragify.
        6. **Fecha de último acceso**.
        7. **Preferencias de visualización** (tema oscuro/claro, idioma, métricas favoritas a mostrar).
        8. **Estado de verificación de cuenta** mediante Steam OAuth (verificado/no_verificado).
        9. Un campo booleano indicando si ha proporcionado la **clave de juego única** para acceso a datos completos.
        10. **Estado de la cuenta** (activa, suspendida, eliminada).
    2. Esta entidad tiene una **relación uno a uno (o cero a uno)** con **JUGADORES_CS2**, ya que un usuario de Fragify puede opcionalmente vincular una única cuenta de Steam/CS2, y un jugador de CS2 solo puede estar vinculado a un único usuario de Fragify.
    3. Tiene una **relación uno a muchos** con la futura entidad **REPORTES**, donde un usuario de Fragify puede realizar múltiples reportes de conducta (**ID_Usuario_Fragify** actuará como clave foránea en REPORTES para identificar al reportante).
    4. Esta entidad **no se implementa en la fase actual** del backend pero está contemplada en el diseño para facilitar su incorporación futura.
2. **Entidad REPORTES (Futura)**: Esta es una de las **entidades más críticas** para la funcionalidad futura de colaboración con Valve, pero **no se implementará** hasta que exista el FRONTEND que permite a usuarios autenticados realizar reportes.
    1. Los **atributos** planificados son:
        1. **Identificador único** del reporte (**ID_Reporte**).
        2. La **clave foránea** del usuario de Fragify que realiza el reporte (**ID_Usuario_Fragify**).
        3. La **clave foránea** del jugador de CS2 reportado (**ID_Jugador_CS2_Reportado**).
        4. La **clave foránea** de la partida asociada al reporte (**ID_Partida**).
        5. El **código de compartición de demo** de esa partida.
        6. **Fecha y hora exacta** en que se realizó el reporte.
        7. **Tipo de infracción reportada** (cheating mediante Wallhack, cheating mediante Aimbot, griefing, toxicidad verbal, abandono intencional, etc.)
        8. Una **descripción opcional adicional** del reportado.
        9. El **estado del reporte** (pendiente, en_revision, resuelto).
        10. **Campos de auditoría**.
    2. Esta entidad tendría **relaciones** con:
        1. **USUARIOS_FRAGIFY** (para el reportante) con **relación uno a muchos**.
        2. **JUGADORES_CS2** (para el reportado) con **relación uno a muchos**.
        3. **PARTIDAS** (para la partida asociada al reporte) con **relación uno a muchos**.
    3. Aunque **no se implementa en la fase actual**, su diseño está contemplado para facilitar su futura incorporación sin modificaciones estructurales importantes.

> Todas estas entidades están diseñadas para **mantener la coherencia de los datos** mediante **claves primarias únicas** y **claves foráneas** que establecen las relaciones necesarias, garantizando que **no existen datos huérfanos** y que toda la información esté correctamente vinculada dentro del sistema. El diseño contempla la **escalabilidad necesaria** para incorporar las funcionalidades futuras sin requerir modificaciones estructurales importantes.

## 3.2 Requisitos de integridad de datos

Para asegurar la **consistencia, precisión y confiabilidad** de los datos almacenados en la BBDD, el sistema debe implementar las siguientes **restricciones, reglas de validación y mecanismos de integridad**:

1. **Integridad de entidad:** Todas las tablas deben tener definida una **clave primaria única** que identifique de forma inequívoca cada registro. Por ejemplo, **ID_Jugador_CS2** en la tabla **JUGADORES_CS2**, **ID_Partida** en la tabla **PARTIDAS**, **ID_Ranking_PREMIERE** en la tabla **RANKING_PREMIERE**, etc.
    1. Estas claves primarias **no pueden contener valores nulos** y deben ser **únicas** en toda la tabla, evitando cualquier duplicidad de registros.
    2. Se recomienda utilizar **claves primarias autoincrementales** de tipo entero para optimizar el rendimiento en las consultas y las relaciones entre tablas.
2. **Integridad referencial**: Las **claves foráneas** deben mantener la **consistencia de las relaciones** entre tablas, garantizando que no existan referencias a registros inexistentes.
    1. Por ejemplo:
        1. En la tabla **ESTADISTICAS_JUGADOR**, el campo **ID_Jugador_CS2** debe hacer referencia obligatoriamente a un ID_Jugador_CS2 existente en la tabla **JUGADORES_CS2**.
        2. En la tabla **PARTIDAS**, el campo **ID_Servidor** debe referenciar a un servidor existente en la tabla **SERVIDORES**.
    2. Si se intentara eliminar un jugador que tiene estadísticas, partidas o rankings asociados, el sistema debe implementar **políticas apropiadas**:
        1. Para datos históricos críticos se debe usar **RESTRICT** para prevenir la eliminación accidental, o implementar **eliminación lógica (soft delete)** mediante un campo de estado en lugar de eliminación física.
        2. Para la futura entidad **REPORTES**, **no se debe permitir la eliminación física** de jugadores reportados ya que esta información será crítica para Valve.
3. **Unicidad de campos específicos:** Ciertos campos deben garantizar **valores únicos** en toda la tabla para evitar conflictos y duplicidades.
    1. El **Steam ID** de cada jugador debe ser **único** en la tabla **JUGADORES_CS2**, ya que cada cuenta de Steam es única y no pueden existir dos jugadores con el mismo Steam ID.
    2. El **código de compartición de demo** debe ser **único** en la tabla **PARTIDAS**, puesto que cada partida de CS2 genera un código irrepetible que identifica su demo.
    3. Si en el futuro se almacena un **identificador de partida proporcionado por Valve**, este también debe ser **único**.
    4. Cualquier intento de insertar valores duplicados en estos campos debe ser rechazado automáticamente por el sistema mediante **restricciones de unicidad (UNIQUE constraints)**.
4. **Validación de tipos de datos y formatos**: El sistema debe validar que los datos introducidos cumplan con el **tipo y formato esperado** específico de Counter-Strike 2 y Steam.
    1. Los **Steam IDs** deben seguir el **formato estándar de Steam** (STEAM_0:X:XXXXXXXX) o el **formato moderno STEAMID64** (7656119XXXXXXXXXX).
    2. Los **códigos de compartición de demos** deben tener el **formato específico de CS2** (CSGO-XXXXX-XXXXXXXXXX-XXXXX-XXXXX donde X son carácteres alfanuméricos).

    3. Las **direcciones IP de servidores** deben cumplir con el **formato IPv4 válido** (XXX.XXX.XXX.XXX).
    4. Las **fechas** deben estar en formato válido y **no pueden ser fechas futuras** para registros históricos como partidas jugadas.
    5. Los **valores numéricos** como eliminaciones, muerte, puntos de ELO o MVP deben ser **enteros no negativos** (mayor o igual a cero).
5. **Validación de rangos y valores permitidos**: Ciertos campos tienen **rangos de valores válidos** que deben ser respetados mediante **restricciones CHECK** o validaciones en la lógica de inserción.
    1. Los **puntos de ELO en PREMIERE** deben ser valores enteros no negativos, típicamente **entre 0 y 40000** (aunque teóricamente no tienen límite superior establecido por Valve, al menos por el momento).
    2. El **ranking competitivo por mapa** solo puede tomar valores de la **escala oficial de Counter-Strike 2**: **Silver I**, **Silver II**, **Silver III**, **Silver IV**, **Silver Elite**, **Silver Elite Master**, **Gold Nova I**, **Gold Nova II**, **Gold Nova III**, **Gold Nova Master**, **Master Guardian I**, **Master Guardian II**, **Master Guardian Elite**, **Distinguished Master Guardian**, **Legendary Eagle**, **Legendary Eagle Master**, **Supreme Master First Class**, **Global Elite**.
    3. El **nombre del mapa** debe limitarse a los **mapas oficiales del pool competitivo actual**.
    4. Los **porcentajes** como precisión de disparo o ratio de headshots deben estar **entre 0 y 100**.
    5. El **modo de juego** debe ser específicamente **"Competitivo MM"** o **"PREMIERE"**.
6. **Reglas de negocio específicas**: El sistema debe implementar reglas de validación que reflejen la **lógica del negocio** y del **juego CS2**.
    1. El **ratio K/D** debe calcularse automáticamente como **eliminaciones divididas por muertes**, con manejo especial cuando las muertes son 0 (estableciendo el valor como las eliminaciones totales o un valor por defecto como **NULL**).
    2. El **porcentaje de victorias** debe calcularse automáticamente como **(partidas_ganadas / partidas_totales) multiplicado por 100**.
    3. La **duración de una partida competitive o PREMIERE** debe estar dentro de **rangos razonables** (típicamente **entre 20 y 90 minutos**), valores fuera de este rango deberían generar advertencias, al menos que uno de los equipos se haya rendido.
    4. El **resultado de una partida** expresado como puntuación debe ser coherente con el **sistema de CS2**: En el sistema actual **(MR12)** el primer equipo en llegar a **13 rondas** gana en modo regulación, pero pueden existir resultados como **13-11**, **13-10**, etc, e incluso resultados de tiempo extra.
    5. **No se debe permitir** que la fecha de una partida sea **posterior a la fecha actual del sistema**.
7. **Triggers y procedimientos de validación automática**: Se deben implementar **triggers** en la base de datos que ejecuten **validaciones y cálculos automáticos** antes y después de intentar o actualizar datos:
    1. Un **trigger** que actualice automáticamente las **estadísticas agregadas** en **ESTADISTICAS__JUGADOR** cuando se inserte un nuevo registro en **RENDIMIENTO_PARTIDA**, recalculando los **totales acumulados**, el **ratio K/D**, y el **porcentaje de victorias**.
    2. Un **trigger** que capture **instantáneas periódicas** del ranking PREMIERE en la tabla **HISTORICO_RANKING_PREMIERE** cuando se detecten **cambios significativos** en los **puntos de ELO** de un jugador.
    3. Un **trigger** que valide que al insertar un registro en **RENDIMIENTO_PARTIDA**, tanto el **jugador** como la **partida existan** en sus respectivas tablas.
    4. Un **trigger** que actualice automáticamente la **fecha de última actualización de datos** cuando se modifiquen registros de jugadores o estadísticas.
8. **Valores por defecto y campos obligatorios**: Se deben establecer claramente qué campos son **obligatorios (NOT NULL)**, y cuáles pueden ser **opcionales**, así como definir **valores por defecto apropiados**.

    1. Campos críticos como **Steam ID**, **ID de partida**, **código de compartición de demo**, **fecha de partida**, **modo de juego**, **puntos de ELO en PREMIERE** son **obligatorios** y **no pueden ser nulos**.
    2. Campos como **descripción adicional**, **nombre de usuario de Steam** (que puede obtenerse posteriormente), **latencia promedio de servidor** pueden ser **opcionales** inicialmente.
    3. Los **valores por defecto** deben definirse donde sea lógico:
        1. **Fecha de registro** de un jugador será la **fecha actual del sistema (CURRENT_TIMESTAMP)**.
        2. **Estado** de un servidor nuevo será **"activo"**.
        3. **Valores numéricos de estadísticas iniciales** serán **0** cuando se cree un nuevo registro de jugador.
9. **Consistencia de datos agregados**: Dado que la BBDD almacenará tanto **datos detallados de partidas individuales** como **estadísticas agregadas de jugadores**, es fundamental mantener la **consistencia entre ambos niveles**.
    1. Se debe implementar un mecanismo que garantice que las **estadísticas totales** en **ESTADISTICAS_JUGADOR** coincidan exactamente con la **suma de los rendimientos individuales** en **RENDIMIENTO_PARTIDA** para ese jugador. Esto puede lograrse mediante **triggers automáticos** que recalculen las estadísticas agregadas, o mediante **procedimientos de validación periódicos** que detecten y corrijan inconsistencias.
    2. Se debe registrar en un **log** cualquier **inconsistencia detectada** para investigación posterior.
10. **Validación de códigos de compartición de demos**: Los **códigos de compartición de demos** son críticos para el futuro **sistema de reportes**, ya que permitirán a los **inspectores de Overwatch** acceder directamente a las **repeticiones de partidas**.
    1. Se debe implementar una **validación robusta del formato** de estos códigos siguiendo el **estándar de CS2**.
    2. Además, se debe considerar que estos códigos tiene una **validez temporal limitada** (típicamente **30 días** desde que se jugó la partida según las políticas de Valve), por lo que se podría incluir un campo que indique si el código **sigue siendo válido o ha expirado**, aunque esto requeriría **actualizaciones periódicas**.

> Todas estas medidas de integridad son fundamentales para garantizar que la información almacenada en el backend sea **confiable**, **consistente** y **útil** tanto para el análisis actual como para las futuras implementaciones de la **plataforma web Fragify** y el **sistema de reportes colaborativo con Valve Corporation**.

### 3.3 Requisitos de Seguridad de Datos

Aunque muchas funcionalidades de seguridad relacionadas con **autenticación de usuarios** dependen del desarrollo futuro del **FRONTEND**, es fundamental que la **BBDD backend** se diseñe desde el inicio con **medidas de seguridad robustas** que protejan la **confidencialidad**, **integridad** y **disponibilidad de los datos**:

1. **Control de acceso basado en roles (diseño preparatorio)**: El sistema debe implementar un **modelo de control de acceso diferenciado** preparado para los diferentes tipos de usuarios que interactuarán con la base de datos en el futuro.
    1. Se establecerán los siguientes **roles principales:**
        1. **Administrador_Sistema**: Tendrá **permisos completos** de lectura, escritura, actualización y eliminación sobre todas las tablas para **tareas de mantenimiento del sistema**, gestión de la BBDD y **resolución de incidencias técnicas**.
        2. **Servicio_API_Recopilacion**: Un **usuario técnico** con permisos específicos para que el **servicio automatizado de recopilación de datos** pueda insertar y actualizar información de jugadores,

partidas, estadísticas y rankings, pero **sin acceso a funcionalidades de eliminación** ni a tablas sensibles futuras como **REPORTES**.

3. **Inspector_Valve** (futuro): Tendrá **permisos de solo lectura (SELECT)** sobre las tablas **REPORTES**, **JUGADORES_CS2**, **PARTIDAS** y tablas relacionadas necesarias para sus **inspecciones de Overwatch**, pero **sin capacidad de modificar, eliminar o insertar datos**.
4. **Usuario_Fragify** (futuro), que podrá **consultar únicamente sus propias estadísticas** y datos mediante la aplicación web, **sin acceso directo a la BBDD**.

2. **Autenticación de acceso a la BBDD**: Todos los **accesos directos a la BBDD** deben estar protegidos mediante **mecanismos de autenticación seguros**.
    1. Los **Administradores del sistema** deben utilizar **credenciales robustas** con **contraseñas fuertes** que cumplan políticas de complejidad (**mínimo 18 caracteres**, combinación de mayúsculas, minúsculas, números y símbolos).
        1. Sólo podrán almacenar/apuntar sus contraseñas en un **gestor de contraseñas fiable** como **KeePassXC**, o en **llaveros de claves SSH** (en caso de que sean compatible con el proyecto, aún por determinar).
        2. Se recomienda implementar **autenticación de dos factores** para accesos administrativos críticos.
    2. El **servicio API de recopilación de datos** debe autenticarse mediante un **usuario técnico** con contraseña almacenada de forma segura en **variables de entorno** o **sistemas de gestión de secretos**, **nunca en código fuente**.
    3. Para el futuro **acceso de Valve Corporation** al sistema de reportes, se proporcionará una **clave API única y segura** con **permisos de solo lectura**, que será un **token criptográficamente seguro** con longitud mínima de **256 bits**, generando mediante **algoritmos seguros de generación de números aleatorios**.
        1. Esta clave podrá ser **revocada y regenerada** por los administradores en caso de compromiso.
3. **Cifrado de datos**: Los **datos sensibles** deben ser protegidos mediante **cifrado** tanto en **reposo** como en **tránsito**.
    1. Todas las **comunicaciones** entre aplicaciones y la BBDD deben realizarse mediante **conexiones cifradas TLS 1.3 o superior**, garantizando que los datos no puedan ser interceptados durante la transmisión.
    2. Las **contraseñas de los usuarios administradores del sistema** deben almacenarse utilizando **algoritmos de hashing seguro** como **bcrypt**, **Argon2** (se recomienda **KDBX 4**) o **PBKDF2** con **rondas de transformación no menores a 100**, **nunca en texto plano** ni con cifrado reversible.
    3. Cuando en el futuro se almacenen **datos personales de usuarios de Fragify** (como direcciones de email), estos deben ser cifrados en la BBDD utilizando **algoritmos de cifrado simétrico AES-256**.
    4. La **clave API de Valve** debe ser almacenada de forma cifrada en la configuración del sistema y solo descifrada en memoria durante su uso.
    5. Para **el cifrado de datos en reposo**, se recomienda utilizar **cifrado a nivel de BBDD (Transparent Data Encryption)** o **cifrado de disco completo** en el servidor que aloja la BBDD.
4. **Auditoría y trazabilidad**: El sistema debe mantener un **registro detallado** de todas las **actividades críticas** que ocurran en la BBDD para garantizar la **trazabilidad** y facilitar la **detección de accesos no autorizados** o comportamientos anómalos.
    1. Se debe crear una tabla **LOG_AUDITORIA** que registre cada operación importante realizada en la BBDD, incluyendo:
        1. **Usuario o servicio** que realizó la operación.
        2. **Fecha y hora exacta** con precisión de milisegundos.
        3. **Tipo de operación** realizada (**INSERT**, **UPDATE**, **DELETE**, **SELECT** en tablas sensibles).

        4. **Tabla y registros específicos** afectados.
        5. **Dirección IP** desde donde se realizó la operación si está disponible.
        6. **Resultado de la operación** (éxito o fallo).
    2. Los **intentos de acceso fallidos** deben registrarse de forma especial con **mayor nivel de detalle** para detectar posibles **intentos de intrusión** o **ataques de fuerza bruta**.
    3. Todos los **accesos futuros de Valve** mediante su clave API deben quedar **registrados en el sistema de auditoría**.
    4. Los **logs de auditoría** deben conservarse durante un **mínimo de 2 años** para cumplir con posibles requisitos legales y de cumplimiento, y **no deben poder ser modificados o eliminados o visualizados** por usuarios regulares, **solo por administradores de sistema** mediante procedimientos específicos, justificados, y también auditados.
5. **Cumplimiento del RGPD**: Aunque en la fase actual del backend **no se almacenan datos personales identificables** de usuarios finales (solo **Steam IDs** y **nombres de usuarios públicos de Steam**), el diseño debe contemplar el cumplimiento del **Reglamento General de Protección de Datos** para cuando se implementen las funcionalidades futuras.
    1. Los **Steam IDs** y **nombres de usuario** son considerados **datos personales bajo el RGPD** al poder identificar a individuos.
    2. Se debe **documentar claramente**:
        1. **Qué datos se recopilan** y **para qué propósito** (análisis de estadísticas de juego y futuro sistema de reportes).
        2. La **base legal del tratamiento** (**consentimiento explícito** de usuarios cuando se registren en Fragify, o **interés legítimo** para datos públicos de Steam).
        3. **Periodo de retención de datos**.
    3. El sistema debe diseñarse de forma que facilite en el futuro la implementación del **derecho al olvido**, permitiendo que usuarios soliciten la **eliminación de sus datos personales**, aunque para el sistema de reportes los datos deberían mantenerse de forma **anonimizada** para preservar la integridad del sistema de moderación.
    4. Se debe implementar **minimización de datos**, almacenando únicamente la información **estrictamente necesaria** para el funcionamiento del servicio.
    5. Se debe documentar y planificar cómo se implementarán en el futuro los **derechos de acceso**, **rectificación**, **portabilidad** y **oposición** que otorga el RGPD a los individuos.
6. **Prevención de inyección SQL y ataques comunes**: Aunque la BBDD backend **no tiene interacción directa con usuarios finales** en esta fase, es fundamental protegerla contra ataques cuando se desarrolle el **FRONTEND**.
    1. Todas las **consultas a la BBDD** desde cualquier aplicación deben utilizar **consultas parametrizadas** o **prepared statements** para prevenir **ataques de inyección SQL**.
    2. **Ninguna entrada de usuario** debe ser concatenada directamente en consultas SQL.
    3. El **servicio API de recopilación de datos** debe **validar y sanitizar** todas las entradas obtenidas de fuentes externas antes de insertarlas en la BBDD.
    4. Se deben establecer **límites de tasa (rate limiting)** en las operaciones de inserción y actualización para prevenir **ataques de denegación de servicio** que intenten sobrecargar la BBDD con consultas masivas.
    5. El sistema debe implementar **validación de entrada robusta** que rechace cualquier dato que no cumpla exactamente con el formato esperado.
7. **Gestión segura de credenciales y secretos**: Los **credenciales de acceso a la BBDD**, **claves API** y otros **secretos** no deben estar almacenados en el código fuente ni en archivos de configuración versionados en el repositorio GitHub.

    1. Se debe utilizar un **sistema de gestión de secretos apropiados** como **variables de entorno seguras**, **HashiCorp Vault**, **AWS Secrets Manager**, **Azure Key Vault**, o similar según la infraestructura utilizada.
    2. Las **credenciales deben rotarse periódicamente** como medida preventiva:
        1. **Contraseñas de administradores** cada **90 días**.
        2. **Credenciales de servicios técnicos** cada **180 días**.
        3. Futura **clave API de Valve** cada **6 meses**.
    3. Debe existir un **procedimiento documentado** para la **revocación inmediata de credenciales** en caso de compromiso, incluyendo **contactos de emergencia** y pasos específicos a seguir.
8. **Backup cifrado**:
    1. Todos los **backups de la BBDD** deben estar **cifrados** antes de su almacenamiento para proteger la información en caso de que los medios de backup sean accedidos de forma no autorizada.
    2. El **cifrado de backups** debe usar **claves diferentes** a las utilizadas para cifrado de datos en producción, y estas claves deben gestionarse de forma segura y estar disponibles para **procesos de recuperación autorizados**.
    3. Los **backups** deben almacenarse en ubicaciones con **controles de acceso físico y lógico estrictos**.
9. **Preparación para seguridad futura**:
    1. El diseño de seguridad debe contemplar que cuando se implemente el **FRONTEND** con **autenticación de usuarios mediante Steam OAuth** y **sincronización con claves de juego**, será necesario implementar:
        1. **Sesiones seguras**.
        2. **Tokens de autenticación** (como **JWT**).
        3. **Protección contra ataques CSRF y XSS**.
        4. **Validación robusta de permisos** en cada operación.

> **La arquitectura de BBDD debe facilitar la implementación de estos mecanismos sin requerir cambios estructurales importantes.**

## 3.4 Requisitos de Mantenimiento de Datos

Para garantizar la **continuidad**, **disponibilidad**, **calidad** y **rendimiento del sistema** a largo plazo, se deben establecer **políticas y procedimientos claros de mantenimiento de datos** que cubran la **actualización**, **eliminación**, **respaldo**, **archivado** y **optimización de la información**:

1. **Actualización automatizada de datos**: El sistema debe contar con **mecanismos automatizados** para mantener la información actualizada mediante la **integración con APIs de CS2**.
    1. El **servicio de recopilación de datos** debe ejecutarse de forma **periódica** para obtener nuevas partidas de los jugadores registrados en la BBDD, con una **frecuencia de actualización configurable** según las limitaciones de la API utilizada y los recursos disponibles (recomendado: **cada 30 minutos** para jugadores activos recientemente y **cada 6 horas** para jugadores menos activos).
    2. Cuando se detecten **nuevas partidas** para un jugador, el sistema debe automáticamente **insertar los registros correspondientes** en las tablas **PARTIDAS** y **RENDIMIENTO_PARTIDA**, y **actualizar las estadísticas agregadas** en **ESTADISTICAS_JUGADOR** y **RANKING_PREMIERE** mediante **triggers** o **procedimientos almacenados** que recalculen los totales acumulados, ratios y porcentajes.
    3. La **captura de instantáneas históricas** en **HISTORICO_RANKING_PREMIERE** debe ejecutarse **semanalmente** mediante un proceso automatizado que registre el estado actual del ranking de cada jugador activo.
    4. Los datos de **servidores** como latencia promedio y estado deben actualizarse **diariamente** o según disponibilidad de información.

2. **Actualización manual supervisada**: Debe existir un **proceso documentado** para que administradores puedan **corregir manualmente datos erróneos o inconsistentes** que puedan surgir debido a fallos en las APIs externas, errores de sincronización, o bugs en el sistema de recopilación.
    1. Cada **corrección manual** debe quedar **registrada en el sistema de auditoría** incluyendo:
        1. El **administrador** que la realizó.
        2. La **fecha**.
        3. Los **datos modificados**.
        4. La **justificación del cambio**.
    2. Se debe establecer un **protocolo de revisión periódica (mensual)** donde se ejecuten **consultas de validación** que detecten posibles inconsistencias como estadísticas agregadas que no coinciden con la suma de rendimientos individuales, **registros huérfanos**, **fechas futuras**, o **valores fuera de rangos esperados**.
3. **Políticas de eliminación de datos**: Se deben establecer **políticas claras** sobre **cuándo y cómo eliminar información** de la BBDD, considerando tanto **requisitos técnicos** como **legales**.
    1. Los **datos de jugadores de CS2 obtenidos de fuentes públicas** (**Steam IDs**, **estadísticas de juego**) **no requieren eliminación obligatoria** ya que son **datos públicos del juego**, pero se puede establecer una **política de limpieza de jugadores inactivos**:
        1. Si un jugador **no ha jugado ninguna partida registrada** en los **últimos 3 años** y **no tiene reportes asociados** (cuando se implemente esta funcionalidad), sus datos pueden ser **archivados o eliminados** previo análisis de uso.
    2. Las **partidas de con más de 3 años de antigüedad**:
        1. Pueden ser candidatas para **archivar**, especialmente considerando que los **códigos de compartición de demos expiran** aproximadamente **30 días** después de jugarse, por lo que mantener estos códigos a largo plazo tiene utilidad limitada.
        2. Sin embargo:
            1. Las **estadísticas agregadas** y los datos de **RENDIMIENTO_PARTIDA** deben **conservarse** para mantener el **historial completo de rendimiento** de los jugadores.
    3. **Para el futuro sistema de reportes**:
        1. Los **datos de reporte** deben conservarse de forma **indefinida** mientras estén **pendientes o en revisión**, y durante **al menos 5 años** después de su resolución para mantener un **historial útil de moderación** para Valve.
4. **Eliminación segura y RGPD**: Cuando en el futuro se almacenen **datos personales de usuarios de Fragify** y se reciban **solicitud de ejercicio del derecho al olvido del RGPD**, se debe implementar un **proceso de eliminación segura** que **sobrescriba los datos personales identificables** pero **mantenga la integridad del sistema**.
    1. Para **reportes realizados por usuarios que soliciten eliminación de cuenta**:
        1. Se debe **anonimizar su identidad** reemplazando el **ID del usuario** por un **identificador genérico** que contenga el **Steam ID**, pues es público **"Usuario_Eliminado_XXXX"** (donde XXXX será el **SteamID64**, necesario para mantener la integridad del sistema) mientras se conserva el reporte en sí para Valve.
        2. La **eliminación** debe realizarse mediante **procedimientos de borrado seguro**, **no simplemente marcando registros como eliminados**, y debe quedar **auditada** con timestamp y justificación legal.
    2. Para **reportes realizados por usuarios que no soliciten eliminación de cuenta**:
        1. Se debe mantener la **anonimización del usuario**, pero se puede conservar su **ID original** en un registro separado para fines de auditoría.

    2. La **eliminación** debe realizarse de la misma manera que para los usuarios que solicitan eliminación de cuenta.
5. **Sistema de backup**: El sistema de backup **solo es útil si realmente funciona cuando se necesita**.
    1. Se deben realizar **pruebas periódicas de recuperación** (al menos **trimestralmente**, idealmente **cada 2 meses**) donde se verifique que los backups son **válidos**, **completos** y **restaurables**.
        1. Estas pruebas deben:
            1. Realizarse en un **entorno de prueba aislado**, **nunca en producción**.
            2. Documentar el **tiempo de recuperación observado** (**Recovery Time Objective - RTO**) y el **punto de recuperación alcanzado** (**Recovery Point Objective - RPO**).
                1. El **RTO objetivo** debe ser **inferior a 4 horas** para recuperación completa del servicio.
                2. El **RPO** no debe superar las **6 horas** de pérdida de datos en el peor escenario (idealmente menor si los backups incrementales funcionan correctamente).
            3. Los **resultados de estas pruebas** deben documentarse y cualquier problema detectado debe **resolverse inmediatamente**.
6. **Archivado de datos históricos**: Para **optimizar el rendimiento del sistema** conforme la BBDD crezca en tamaño se debe implementar una **estrategia de archivado de datos históricos** que mantenga la información disponible pero en un **almacenamiento secundario** optimizado para consultas menos frecuentes.
    1. Los **datos de RENDIMIENTO_PARTIDA** de partidas con **más de 2 años de antigüedad** pueden ser **migrados a tablas de archivo** o a una **BBDD secundaria de históricos**, manteniendo disponibles en las tablas principales únicamente los **datos agregados** en **ESTADISTICAS_JUGADOR** y las entradas reciente.
    2. Los **registros de auditoría** con **más de 1 año** pueden ser **comprimidos** y movidos a **sistemas de almacenamiento de largo plazo** con acceso menos frecuente pero más económico.
    3. Los **códigos de compartición de demos** de partidas con **más de 60 días** pueden ser **eliminados** ya que Valve no mantiene las demos disponibles más allá de aproximadamente **30 días**, conservando solo la información de que existió la partida pero eliminando el campo de código ya inútil.
    4. El **sistema de archivado** debe mantener la **integridad referencial** cuando sea posible, o implementar **vistas** que combinen transparentemente datos activos y archivados para **consultas históricas completas**.
    5. Los **datos archivados** deben incluirse en el **sistema de backups** y seguir las mismas **políticas de seguridad** que los datos activos.
7. **Optimización de rendimiento**: Se debe implementar un **programa de mantenimiento preventivo** que garantice que el **rendimiento de la base de datos** se mantenga óptimo conforme crece.
    1. El **tamaño de la base de datos** debe **monitorizarse continuamente** y proyectar cuándo será necesario **ampliar el almacenamiento** o implementar **estrategias de particionado de tablas**.
    2. Los **tiempos de respuesta de las consultas más frecuentes** (obtener estadísticas de un jugador, listar partidas recientes, consultar ranking PREMIERE) deben ser **medidos continuamente** mediante herramientas de monitorización, y si se detecta **degradación del rendimiento por encima del 20%** respecto a la línea base, se debe **investigar inmediatamente** la causa (falta de índices, consultas ineficientes, fragmentación, crecimiento excesivo de tablas).
    3. La **fragmentación de tablas e índices** debe ser analizada **mensualmente**, y cuando supere el **30%** se debe ejecutar **desfragmentación** mediante operaciones **REBUILD** o **REORGANIZE** según el motor de base de datos utilizado.
    4. Las **estadísticas del optimizador de consultas** deben actualizarse **semanalmente** (comando **UPDATE STATISTICS** o equivalente) para garantizar que el motor genere **planes de ejecución eficientes**

basados en la distribución actual de datos.

8. **Índices y optimización de consultas**:
    1. **Se deben crear índices apropiados** en los **campos más utilizados** en consultas y joins:
        1. **Índice único** en **Steam_ID** de **JUGADORES_CS2**.
        2. **Índice** en **fecha de partidas** para consultas temporales.
        3. **Índice compuesto** en **(ID_Jugador_CS2, fecha)** en **RENDIMIENTO_PARTIDA** para consultas de historial de jugador.
        4. **Índice** en **puntos de ELO** en **RANKING_PREMIERE** para consultas de clasificación.
    2. **Se debe evitar la sobre-indexación** que puede degradar el rendimiento de las operaciones de escritura.
    3. **Las consultas SQL deben ser revisadas** periódicamente para identificar **cuellos de botella**, utilizando **herramientas de análisis de planes de ejecución** (**EXPLAIN** en MySQL/PostgreSQL, **Execution Plan** en SQL Server) para **optimizar consultas lentas**.
    4. **Se recomienda implementar caché de consultas frecuentes** en el nivel de aplicación para reducir la carga sobre la base de datos.
9. **Monitorización continua del sistema**: Se debe implementar un sistema de monitorización que alerte a los administradores sobre problemas potenciales antes de que afecten al servicio.
    1. Por ejemplo, utilizando herramientas como Prometheus, Grafana, Zabbix, o servicios en la nube como AWS CloudWatch.
    2. Las **métricas claves a monitorizar** incluyen:
        1. Uso de CPU y memoria del servidor de BBDD.
        2. Espacio en disco disponible con alertas cuando se supere el 70% de capacidad.
        3. Número de conexiones activas a la BBDD con alertas si se acerca al límite configurado.
        4. Tiempos de respuesta promedio de las consultas principales con alertas si superan los umbrales establecidos (por ejemplo 2 segundos para consultas de estadísticas).
        5. Tasa de errores en operaciones de BBDD.
        6. Tamaño de las tablas principales y su crecimiento mensual proyectado.
        7. Disponibilidad del servicio de BBDD.
    3. **Las alertas deben configurarse** para notificar a los administradores vía email o sistemas de mensajería inmediata cuando se detecten anomalías, permitiendo respuesta rápida antes de que los problemas escalen.
10. **Ventanas de mantenimiento**:
    1. Se deben programar ventanas de mantenimiento periódicas donde se realicen tareas de optimización que puedan requerir que la BBDD esté fuera de línea o con rendimiento reducido.
        1. **Se recomienda establecer una ventana mensual** los primeros domingos del mes entre las 3:00 y 6:00 AM hora local de servidor, cuando la actividad de usuarios sea mínima.
            1. Durante estas ventanas se pueden realizar tareas como:
                1. Desfragmentación de tablas e índices.
                2. Actualización de estadísticas del optimizador.
                3. Limpieza de logs antiguos.
                4. Verificación de integridad de datos mediante comandos como DBCC CHECKDB en SQL Server o CHECK TABLE en MySQL.
                5. Aplicación de actualizaciones de seguridad del motor de base de datos y pruebas de recuperación de backups.
            2. **Las ventanas de mantenimiento deben:**
                1. **Ser comunicadas con antelación** si hay usuarios o servicios que puedan verse afectados.

2. Su duración y resultados deben ser documentados.

11. **Procedimientos de actualización y migración**: Conforme el proyecto evolucione, será necesario realizar cambios en la estructura de la BBDD para agregar nuevas funcionalidades (como las tablas de USUARIOS_FRAGIFY y REPORTES cuando se desarrolle el FRONTEND).
    1. **Se debe establecer un procedimiento de control de cambios que incluya**:
        1. Documentación detallada de todos los cambios estructurales propuestos.
        2. Análisis del impacto de los cambios en datos existentes y en el rendimiento.
        3. Creación de scripts de migración que puedan ejecutarse de forma automatizada y que sean indempotentes (pueden ejecutarse múltiples veces sin causar errores).
        4. Pruebas exhaustivas de los scripts de migración en entorno de desarrollo y pre-producción antes de aplicarlos a producción.
        5. Backup completo inmediatamente antes de ejecutar cualquier migración en producción.
        6. Plan de rollback documentado en caso de que la migración falle.
    2. Se recomienda utilizar herramientas de gestión de migraciones de BBDD como Flyway, Liquibase o las migraciones integradas en frameworks ORM para mantener control de versiones de la estructura de la BBDD.
12. **Continuidad del negocio y recuperación ante desastres**: En caso de fallo catastrófico del sistema principal, debe existir un plan de continuidad que permita restaurar el servicio en el menor tiempo posible.
    1. Se debe considerar mantener un servidor de base de datos secundario configurado como réplica (hot standby o warm standby) que pueda asumir la carga en caso de fallo del principal, aunque esto dependerá del presupuesto y recursos disponibles para el proyecto.
    2. La infraestructura debe diseñarse minimizando puntos únicos de fallo (Single Point of Failure) con redundancia en componentes críticos como almacenamiento mediante RAID, fuentes de alimentación redundantes, y conectividad de red redundante.
    3. Debe existir documentación actualizada y accesible de todos los procedimientos de recuperación ante desastres, incluyendo:
        1. Diagrama de la infraestructura actual.
        2. Credenciales de acceso de emergencia almacenadas en ubicación segura pero accesible.
        3. Contactos de personal responsable con disponibilidad 24/7.
        4. Pasos detallados para restaurar desde backup.
        5. Pasos para activar servidor secundario si existe.
        6. Procedimientos de validación de integridad después de recuperación.
        7. Tiempos estimados de recuperación para diferentes escenarios.
    4. Esta documentación debe:
        1. Revisarse y actualizarse semestralmente.
        2. Todo el personal técnico clave debe estar familiarizado con los procedimientos mediante simulacros periódicos.
13. **Gestión de crecimiento y escalabilidad**: El sistema debe diseñarse anticipando el crecimiento futuro. Si el proyecto tiene éxito y la base de usuarios de la plataforma Fragify crece significativamente, la BBDD debe poder escalar para manejar el incremento en volumen de datos y carga de consultas.
    1. Se debe evaluar periódicamente (cada 6 meses) si la infraestructura actual es suficiente o si se requiere:
        1. Escalado vertical agregando más recursos (CPU, RAM, almacenamiento más rápido) al servidor existente.
        2. Escalado horizontal implementando réplicas de lectura para distribuir la carga de consultas.
        3. Particionado de tablas grandes por rangos de fechas o por jugadores para mejorar rendimiento.
        4. Implementación de caché distribuida como Redis o Memcached para reducir carga en la BBDD o migración a soluciones de BBDD más escalables si el sistema actual alcanza sus límites.

2. El particionado es especialmente relevante para tablas que crecerán indefinidamente como PARTIDAS y RENDIMIENTO_PARTIDA, donde se podría particionar por año o trimestre para mantener rendimiento óptimo.

14. **Documentación de mantenimiento**:
    1. Todos los procedimientos, políticas y tareas de mantenimiento deben estar completamente documentados en un manual de operaciones que incluya:
        1. Descripción de cada tarea de mantenimiento periódica con su frecuencia.
        2. Pasos detallados para ejecutar cada procedimiento.
        3. Criterios de éxito y validación.
        4. Solución de problemas comunes.
        5. Contactos de escalado en caso de incidencias.
        6. Ubicación de backups y procedimientos de acceso.
        7. Credenciales y sistemas de gestión de secretos.
        8. Herramientas utilizadas y sus configuraciones.
        9. Histórico de cambios importantes en la infraestructura.
    2. Esta documentación debe mantenerse actualizada y ser accesible para todo el equipo técnico autorizado.
        1. Se recomienda implementar un sistema de documentación versionado como wiki interno o repositorio de documentación junto con el código en el repositorio privado del proyecto.

> **Estos requisitos de mantenimiento garantizan** que el sistema CS2-SVSS pueda operar de forma confiable, eficiente y segura a largo plazo, manteniendo la calidad de los datos, el rendimiento adecuado, y la capacidad de recuperarse ante incidentes, estableciendo así una base sólida para el crecimiento futuro del proyecto y la implementación de la web Fragify y el sistema de colaboración con Valve Corporation.

---

# 4. CONCLUSION

En este documento se han detallado los **requisitos funcionales y no funcionales** para el diseño de la **base de datos backend** del **sistema CS2-SVSS**, que servirá como **columna vertebral** para la futura **plataforma web Fragify** y el **sistema de reportes colaborativo** con **Valve Corporation**. Se han establecido **criterios claros** para garantizar la **integridad**, **seguridad**, **mantenimiento** y **escalabilidad** de los datos, asegurando que el sistema pueda **evolucionar** y **adaptarse** a las **necesidades futuras** sin comprometer la **calidad** ni la **confiabilidad** de la información almacenada.

> La implementación rigurosa de estos requisitos es fundamental para el éxito del proyecto, proporcionando una **base sólida** que soporte tanto las **funcionalidades actuales** como las **futuras expansiones previstas** en el roadmap del sistema.